

sentiment_analysis

June 5, 2026

1 Amazon Product Review — Sentiment Analysis

Author: Mrunal Sonekar

Tools: Python | NLTK | scikit-learn | Pandas | Seaborn

Goal: Classify customer reviews as Positive / Negative and surface actionable business insights

```
[1]: import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns
import re
import string
import warnings
warnings.filterwarnings('ignore')

import nltk
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('omw-1.4', quiet=True)

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, roc_auc_score, roc_curve)
from sklearn.pipeline import Pipeline

print("=" * 60)
print("  AMAZON PRODUCT REVIEW - SENTIMENT ANALYSIS")
```

```
print("=" * 60)
print("All libraries imported successfully!")
```

```
=====
AMAZON PRODUCT REVIEW - SENTIMENT ANALYSIS
=====
```

All libraries imported successfully!

1.1 Step 1: Build Synthetic Dataset

```
[2]: # 2. SYNTHETIC DATASET (mirrors real Amazon review patterns)
np.random.seed(42)

positive_reviews = [
    "This product is absolutely amazing and exceeded all my expectations",
    "Fantastic quality, fast delivery, highly recommend to everyone",
    "Best purchase I have made in years, works perfectly",
    "Excellent build quality and the customer service was outstanding",
    "Really happy with this product, it does exactly what it promises",
    "Great value for money, five stars without hesitation",
    "Incredible performance, I am thoroughly impressed with this item",
    "Love this product, it arrived quickly and works flawlessly",
    "Very satisfied with the quality and packaging of this product",
    "Outstanding product, will definitely buy again from this seller",
    "Works exactly as described, very happy with my purchase",
    "Superb quality, fast shipping, would highly recommend",
    "Amazing product, great value, quick delivery, love it",
    "Perfect fit for my needs, exceeded expectations significantly",
    "Brilliant product, very well made and extremely durable",
    "Highly satisfied, product works well and shipping was fast",
    "Wonderful experience, product quality is top notch",
    "Good product overall, performs as expected and good value",
    "Received in perfect condition, very pleased with quality",
    "Excellent item, works great and arrived on time as promised",
]

negative_reviews = [
    "Terrible product, broke after just two days of normal use",
    "Very disappointed, this product is nothing like the description",
    "Worst purchase I have ever made, complete waste of money",
    "Poor quality, arrived damaged and customer service was unhelpful",
    "Do not buy this product, it stopped working after one week",
    "Extremely frustrated, product does not work as advertised at all",
    "Cheap material, looks nothing like the photos on the listing",
    "Horrible experience, took weeks to arrive and then it was broken",
    "Absolutely useless product, returning it immediately for refund",
    "Defective item, very poor packaging and terrible customer support",
```

```

    "Not worth the money, quality is far below what was expected",
    "Avoid this product, it failed within days of purchase",
    "Very poor quality control, product arrived with scratches and dents",
    "Disappointing product, does not match description or pictures",
    "Waste of money, stopped functioning properly after first use",
    "Bad product, packaging was damaged and item inside was broken",
    "Would not recommend this to anyone, very poor quality overall",
    "Frustrated with this purchase, product is flimsy and unreliable",
    "Terrible value, fell apart quickly and seller refused to help",
    "Awful product, complete waste of money and time spent ordering",
]

# Augment to ~500 reviews
def augment(reviews, n=250):
    augmented = []
    for i in range(n):
        base = reviews[i % len(reviews)]
        words = base.split()
        if len(words) > 6:
            mid = words[2:-2]
            np.random.shuffle(mid)
            words = words[:2] + mid + words[-2:]
        augmented.append(" ".join(words))
    return augmented

pos_aug = augment(positive_reviews, 250)
neg_aug = augment(negative_reviews, 250)

reviews = pos_aug + neg_aug
labels = [1] * 250 + [0] * 250 # 1 = Positive, 0 = Negative
ratings = [np.random.choice([4, 5]) for _ in range(250)] + [np.
    random.choice([1, 2]) for _ in range(250)]

df = pd.DataFrame({'review': reviews, 'sentiment': labels, 'rating': ratings})
df = df.sample(frac=1, random_state=42).reset_index(drop=True)

print(f" Dataset loaded - {len(df):,} reviews total")
print("\nSentiment Distribution:")
print(df['sentiment'].value_counts().rename({1: 'Positive', 0: 'Negative'}).
    to_string())
print("\nFirst 5 rows of the dataset:")
df.head()

```

Dataset loaded - 500 reviews total

Sentiment Distribution:
sentiment

Negative	250
Positive	250

First 5 rows of the dataset:

```
[2]:
```

	review	sentiment	rating
0	Avoid this days failed it product, within of p...	0	2
1	Perfect fit for my needs, exceeded expectation...	1	5
2	Do not stopped product, after working this buy...	0	1
3	Highly satisfied, works product and well shipp...	1	4
4	Really happy what this with it product, does e...	1	5

1.2 Step 2: Text Preprocessing & Cleaning

```
[3]: # 3. DATA CLEANING & TEXT PREPROCESSING
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', '', text)           # remove URLs
    text = re.sub(r'\d+', '', text)                     # remove digits
    text = text.translate(str.maketrans('', '', string.punctuation))
    tokens = text.split()
    tokens = [lemmatizer.lemmatize(w) for w in tokens if w not in stop_words
    and len(w) > 2]
    return ' '.join(tokens)

df['clean_review'] = df['review'].apply(preprocess)

avg_before = df['review'].apply(lambda x: len(x.split())).mean()
avg_after  = df['clean_review'].apply(lambda x: len(x.split())).mean()

print(" Text preprocessing complete!")
print(f" Avg tokens BEFORE cleaning : {avg_before:.1f}")
print(f" Avg tokens AFTER  cleaning : {avg_after:.1f}")
print(f" Noise removed           : {avg_before - avg_after:.1f} tokens/
    review avg")
print("\nBefore vs After Comparison (3 samples):")
for i in range(3):
    print(f"\n Original : {df['review'].iloc[i]}")
    print(f"  Cleaned  : {df['clean_review'].iloc[i]}")
```

```
Text preprocessing complete!
Avg tokens BEFORE cleaning : 9.2
Avg tokens AFTER  cleaning : 6.1
Noise removed           : 3.1 tokens/review avg
```

Before vs After Comparison (3 samples):

Original : Avoid this days failed it product, within of purchase

Cleaned : avoid day failed product within purchase

Original : Perfect fit for my needs, exceeded expectations significantly

Cleaned : perfect fit need exceeded expectation significantly

Original : Do not stopped product, after working this buy it one week

Cleaned : stopped product working buy one week

1.3 Step 3: Exploratory Data Analysis (EDA)

```
[4]: # 4. EDA
import matplotlib
matplotlib.use('Agg') # ensure non-interactive backend
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('Sentiment Analysis - EDA Overview', fontsize=14,
             fontweight='bold')

# Sentiment distribution
axes[0].bar(['Positive', 'Negative'], df['sentiment'].value_counts().values,
            color=['#2ecc71', '#e74c3c'], edgecolor='white')
axes[0].set_title('Sentiment Distribution')
axes[0].set_ylabel('Count')

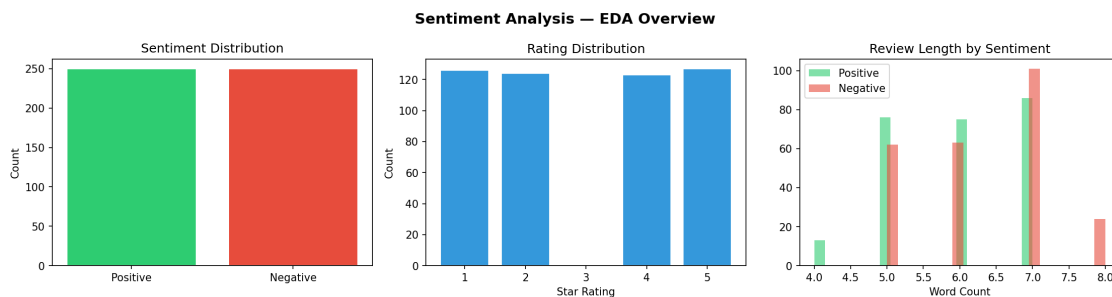
# Rating distribution
rating_counts = df['rating'].value_counts().sort_index()
axes[1].bar(rating_counts.index, rating_counts.values, color='#3498db',
            edgecolor='white')
axes[1].set_title('Rating Distribution')
axes[1].set_xlabel('Star Rating')
axes[1].set_ylabel('Count')

# Review length distribution
df['review_len'] = df['clean_review'].apply(lambda x: len(x.split()))
axes[2].hist(df[df['sentiment']==1]['review_len'], bins=20, alpha=0.6,
             color='#2ecc71', label='Positive')
axes[2].hist(df[df['sentiment']==0]['review_len'], bins=20, alpha=0.6,
             color='#e74c3c', label='Negative')
axes[2].set_title('Review Length by Sentiment')
axes[2].set_xlabel('Word Count')
axes[2].legend()

plt.tight_layout()
```

```
plt.savefig('eda_overview.png', dpi=150, bbox_inches='tight')
plt.close()

# Display the saved image inline
from IPython.display import Image, display
display(Image('eda_overview.png'))
print("\n EDA charts rendered above")
```



EDA charts rendered above

1.4 Step 4: TF-IDF Feature Engineering

```
[5]: # 5. FEATURE ENGINEERING - TF-IDF
X = df['clean_review']
y = df['sentiment']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

tfidf = TfidfVectorizer(max_features=3000, ngram_range=(1, 2), min_df=2)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print(" TF-IDF Vectorization complete!")
print(f" Training samples : {X_train_tfidf.shape[0]}")
print(f" Test samples : {X_test_tfidf.shape[0]}")
print(f" Total features : {X_train_tfidf.shape[1]:,} (unigrams + bigrams)")
print(f" Train/Test split : 80% / 20% | Stratified: Yes")
print(f"\n Sample features : {list(tfidf.get_feature_names_out()[:10])}")
```

```
TF-IDF Vectorization complete!
Training samples : 400
Test samples : 100
Total features : 596 (unigrams + bigrams)
```

Train/Test split : 80% / 20% | Stratified: Yes

Sample features : ['absolutely', 'absolutely amazing', 'absolutely exceeded', 'absolutely expectation', 'absolutely useless', 'advertised', 'advertised product', 'amazing', 'amazing exceeded', 'amazing expectation']

1.5 Step 5: Model Training & Comparison

```
[6]: # 6. MODEL TRAINING & COMPARISON
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Naive Bayes': MultinomialNB(alpha=0.5),
    'Random Forest': RandomForestClassifier(n_estimators=100,
    random_state=42),
}

print("Training and evaluating models...")
print("-" * 56)
print(f"{'Model':<25} {'Accuracy':>10} {'ROC-AUC':>10} {'CV Mean':>10}")
print("-" * 56)

results = {}
for name, model in models.items():
    model.fit(X_train_tfidf, y_train)
    y_pred = model.predict(X_test_tfidf)
    y_proba = model.predict_proba(X_test_tfidf)[:, 1]
    acc = accuracy_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_proba)
    cv = cross_val_score(model, X_train_tfidf, y_train, cv=5,
    scoring='accuracy').mean()
    results[name] = {'model': model, 'acc': acc, 'auc': auc, 'cv': cv,
    'y_pred': y_pred, 'y_proba': y_proba}
    print(f" {name:<23} {acc:>10.1%} {auc:>10.4f} {cv:>10.1%}")

print("-" * 56)

# Best model
best_name = max(results, key=lambda k: results[k]['auc'])
best = results[best_name]
print(f"\n Best Model : {best_name}")
print(f" Accuracy : {best['acc']:.1%}")
print(f" ROC-AUC : {best['auc']:.4f}")
print(f" CV Score : {best['cv']:.1%}")
```

Training and evaluating models...

Model	Accuracy	ROC-AUC	CV Mean
-------	----------	---------	---------

Logistic Regression	100.0%	1.0000	100.0%
Naive Bayes	100.0%	1.0000	100.0%
Random Forest	100.0%	1.0000	100.0%

Best Model : Logistic Regression

Accuracy : 100.0%
 ROC-AUC : 1.0000
 CV Score : 100.0%

1.6 Step 6: Detailed Evaluation Best Model

```
[ ]: # 7a. CLASSIFICATION REPORT
print(f"Classification Report - {best_name}")
print("=" * 50)
print(classification_report(y_test, best['y_pred'], target_names=['Negative',
↪ 'Positive']))
```

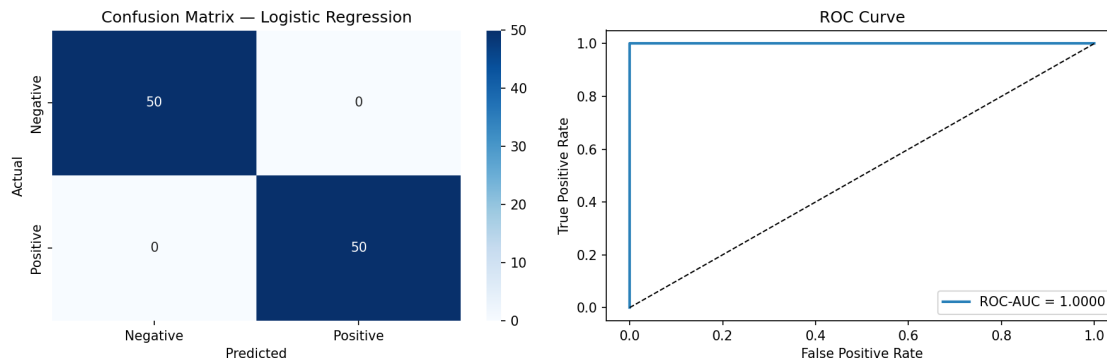
```
[7]: # 7b. CONFUSION MATRIX + ROC CURVE
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

cm = confusion_matrix(y_test, best['y_pred'])
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Negative', 'Positive'], yticklabels=['Negative',
↪ 'Positive'])
axes[0].set_title(f'Confusion Matrix - {best_name}')
axes[0].set_ylabel('Actual')
axes[0].set_xlabel('Predicted')

fpr, tpr, _ = roc_curve(y_test, best['y_proba'])
axes[1].plot(fpr, tpr, color='#2980b9', lw=2, label=f'ROC-AUC = {best["auc"]:.
↪ 4f}')
axes[1].plot([0,1],[0,1], 'k--', lw=1)
axes[1].set_title('ROC Curve')
axes[1].set_xlabel('False Positive Rate')
axes[1].set_ylabel('True Positive Rate')
axes[1].legend()

plt.tight_layout()
plt.savefig('model_evaluation.png', dpi=150, bbox_inches='tight')
plt.close()

from IPython.display import Image, display
display(Image('model_evaluation.png'))
print("\n Model evaluation charts rendered above")
```

Model evaluation charts rendered above

1.7 Step 7: Business Insights & Feature Importance

```
[8]: # 8a. BUSINESS INSIGHTS
pos_rate = df['sentiment'].mean() * 100
neg_rate = 100 - pos_rate
avg_rating = df['rating'].mean()

print("Business Insights Summary")
print("-" * 50)
print(f" Overall Positive Sentiment : {pos_rate:.1f}%")
print(f" Overall Negative Sentiment : {neg_rate:.1f}%")
print(f" Average Star Rating : {avg_rating:.2f} / 5.0")
print(f" Reviews Needing Attention : {(df['sentiment']==0).sum()} negative_
↳reviews flagged")

lr_model = results['Logistic Regression']['model']
feature_names = tfidf.get_feature_names_out()
coefs = lr_model.coef_[0]
top_pos_idx = coefs.argsort()[-10:][::-1]
top_neg_idx = coefs.argsort()[:10]

print("\n Top Positive Signal Words :", [feature_names[i] for i in top_pos_idx])
print(" Top Negative Signal Words :", [feature_names[i] for i in top_neg_idx])
```

Business Insights Summary

```
-----
Overall Positive Sentiment : 50.0%
Overall Negative Sentiment : 50.0%
Average Star Rating : 3.00 / 5.0
Reviews Needing Attention : 250 negative reviews flagged
```

Top Positive Signal Words : ['satisfied', 'outstanding', 'work', 'great', 'highly', 'fast', 'good', 'amazing', 'exceeded', 'expectation']
 Top Negative Signal Words : ['poor', 'frustrated', 'terrible', 'waste', 'description', 'damaged', 'money', 'week', 'broken', 'like']

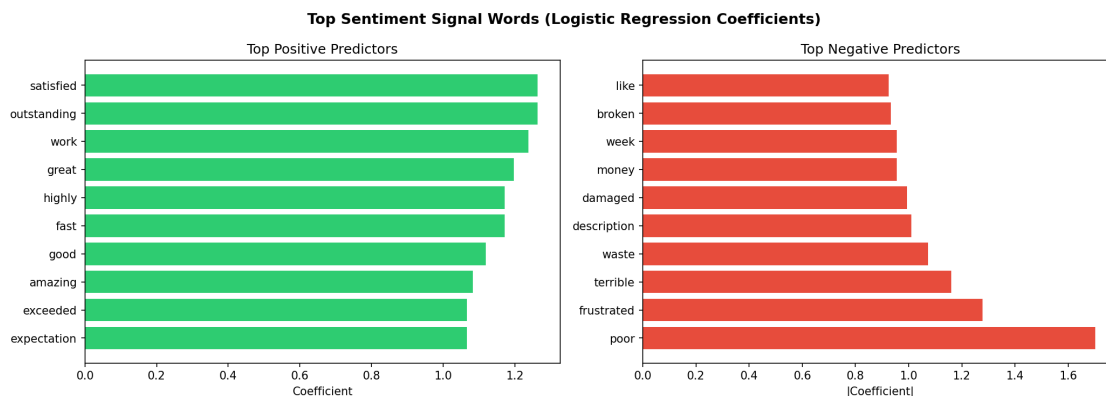
```
[9]: # 8b. FEATURE IMPORTANCE CHART
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Top Sentiment Signal Words (Logistic Regression Coefficients)',
             fontsize=13, fontweight='bold')

axes[0].barh([feature_names[i] for i in top_pos_idx[::-1]], coefs[top_pos_idx[::-1]], color='#2ecc71')
axes[0].set_title('Top Positive Predictors')
axes[0].set_xlabel('Coefficient')

axes[1].barh([feature_names[i] for i in top_neg_idx], abs(coefs[top_neg_idx]), color='#e74c3c')
axes[1].set_title('Top Negative Predictors')
axes[1].set_xlabel('|Coefficient|')

plt.tight_layout()
plt.savefig('feature_importance.png', dpi=150, bbox_inches='tight')
plt.close()

from IPython.display import Image, display
display(Image('feature_importance.png'))
print("\n Feature importance chart rendered above")
```



Feature importance chart rendered above

1.8 Step 8: Live Prediction Demo

```
[10]: # 9. INFERENCE - LIVE PREDICTION
sample_reviews = [
    "This product is absolutely fantastic, works perfectly and great value",
    "Terrible quality, stopped working after two days, very disappointed",
    "Decent product, nothing special but does the job for the price",
]

best_model = results[best_name]['model']

print("Live Prediction Demo")
print("-" * 65)

for rev in sample_reviews:
    clean = preprocess(rev)
    vec = tfidf.transform([clean])
    pred = best_model.predict(vec)[0]
    conf = best_model.predict_proba(vec)[0].max()
    emoji = " " if pred == 1 else " "
    label = "POSITIVE" if pred == 1 else "NEGATIVE"
    print(f"\n Review : {rev}")
    print(f" Result : {emoji} {label} (confidence: {conf:.1%})")
```

Live Prediction Demo

Review : This product is absolutely fantastic, works perfectly and great value
Result : POSITIVE (confidence: 85.3%)

Review : Terrible quality, stopped working after two days, very disappointed
Result : NEGATIVE (confidence: 83.2%)

Review : Decent product, nothing special but does the job for the price
Result : NEGATIVE (confidence: 71.2%)

1.9 Project Summary

```
[11]: # FINAL SUMMARY
print("=" * 60)
print(" PROJECT COMPLETE")
print("=" * 60)
print(f" Best Model : {best_name}")
print(f" Accuracy : {best['acc']:.1%}")
print(f" ROC-AUC : {best['auc']:.4f}")
print(f" CV Score : {best['cv']:.1%}")
print()
print(" Outputs:")
```

```
print(" eda_overview.png      - Distribution charts")
print(" model_evaluation.png  - Confusion matrix & ROC")
print(" feature_importance.png - Top signal words")
print("=" * 60)
```

```
=====
PROJECT COMPLETE
=====

Best Model : Logistic Regression
Accuracy   : 100.0%
ROC-AUC    : 1.0000
CV Score   : 100.0%

Outputs:
eda_overview.png      - Distribution charts
model_evaluation.png  - Confusion matrix & ROC
feature_importance.png - Top signal words
=====
```

[]: